Encoding is essentially the same thing, but meta-characters are completely replaced with an alternative sequence. For example, the < and > characters used to form HTML tags can be encoded: `<script>` (which a browser would interpret as the opening tag of a JavaScript program) can be encoded as `<script>` (which a browser would convert back to the string `<script>` and simply render as text).

Encoding and escape functions are commonly used for the purposes of rendering or transmission across binary-unsafe transport layers. Raw HTML will need to be encoded to another form if it is intended for rendering in a web browser; similarly, some data transfer mechanisms are not 8-bit safe, and binary data needs to be encoded in a specific way to survive a journey (e.g., through email systems). The functions that are specific to these tasks are as follows:

Table 8-20. *Escaping and encoding functions*

`addslashes()`
`stripslashes()`
`addcslashes()`
`stripcslashes()`
	Add/remove backslash char for certain byte values

`base64_encode()` `base64_decode()`	Performs the MIME base64 encode for transport safety
`convert_uuencode()` `convert_uudecode()`	Binary safe conversion of bytes to printable chars (for transmission)
`html_entity_decode()` `htmlentities()`	Convert all applicable chars to/from HTML entities
`htmspecialchars_decode()` `htmlspecialchars()`	Convert a limit set of chars to/from HTML entities
`strip_tags()`	Remove HTML (and PHP) tags from a string
`str_rot13()`	Shift (ASCII) letters by 13 places (aka the rot13 transform)
`quoted_printable_encode()` `quoted_printable_decode()`	Convert string to/from RFC 2045 (Section 6.7) encoding
`quotemeta()`	Add backslash to `.\+*?[]()^$` chars
`mb_decode_numericentity()` `mb_encode_numericentity()`	Multi-byte converts chars to/from HTML numeric entities